A Comprehensive Survey on Adversarial Threats and Defenses in Large Language Models

March 2026

Abstract

The rapid integration of Large Language Models (LLMs) into critical applications has precipitated a paradigm shift in artificial intelligence, simultaneously exposing these advanced systems to a complex and rapidly evolving array of security vulnerabilities. To systematically understand this threat landscape, this survey presents a comprehensive systematic literature review of LLM security, analyzing 29 foundational and contemporary academic papers. The methodology employs a structured categorization of vulnerabilities across the entire model lifecycle, distinctly separating training-time exposures from inference-time attack vectors. Specifically, the review investigates training-time threats, primarily focusing on data and model poisoning techniques that fundamentally compromise the alignment and integrity of foundational models. Furthermore, it explores inference-time vulnerabilities, providing an in-depth analysis of prompt injection, jailbreaking, and sophisticated adversarial attacks that manipulate model outputs. Beyond malicious external exploits, the survey critically examines intrinsic model reliability issues, emphasizing the pervasive challenges of LLM hallucinations and factual inconsistency. As a core practical contribution, this paper evaluates emerging testing methodologies and evaluation frameworks that are essential for rigorously benchmarking model robustness. Additionally, it synthesizes cross-cutting defenses and mitigation strategies designed to neutralize threats across both unimodal and multimodal architectures. By consolidating these diverse research streams, the survey identifies critical gaps in the current literature and outlines pressing open challenges. Ultimately, this work equips researchers and industry practitioners with a unified taxonomy and the actionable insights necessary to design, evaluate, and deploy secure, resilient, and trustworthy generative artificial intelligence systems.

Version 1.0 — March 2026

---

1 Table of Contents

1. 1. Introduction

2. 2. Background and Preliminaries
3. 3. Training-Time Threats: Data and Model Poisoning
4. 4. Inference-Time Threats I: Prompt Injection and Jailbreaking
5. 5. Inference-Time Threats II: Adversarial Attacks
6. 6. Model Reliability: LLM Hallucinations and Factual Consistency
7. 8. Cross-Cutting Defenses and Mitigation Strategies
8. 7. Testing Methodologies and Evaluation
9. 9. Discussion and Open Challenges
10. 10. Conclusion

---

2 1. 1. Introduction

In the past decade, deep neural networks (DNNs) have achieved remarkable milestones, fundamentally transforming domains such as computer vision, natural language processing, and multimodal learning Wang et al. [2024]. However, the artificial intelligence paradigm has recently undergone a massive shift toward foundational Large Language Models (LLMs) and Multimodal Large Language Models (MLLMs). Driven by scaling laws and trained on unprecedented volumes of data, these models exhibit emergent capabilities in reasoning, instruction following, and complex problem-solving. Consequently, they are being rapidly integrated into high-stakes, safety-critical applications, including autonomous driving, medical diagnosis, financial forecasting, automated code generation, and complex recommendation systems. Despite these unprecedented advancements, the widespread deployment of LLMs has exposed a critical and pervasive flaw: they are highly vulnerable to a diverse spectrum of security threats and reliability failures.

While traditional deep learning models have long been known to harbor vulnerabilities—such as adversarial attacks involving imperceptible perturbations to continuous input spaces Wang et al. [2024]—the advent of LLMs has significantly expanded and complicated the attack surface. The natural language interface of LLMs, combined with their opaque, massive-scale training pipelines, introduces novel attack vectors that defy traditional security paradigms. Ensuring the security, alignment, and factual reliability of these models is no longer merely an academic curiosity, but a foundational necessity for developing robust, trustworthy, and secure AI systems.

The LLM security landscape spans the entire model lifecycle, beginning with the training phase. Because LLMs require massive, largely uncurated web-scale datasets for pre-training and fine-tuning, they are inherently susceptible to data and

model poisoning. Adversaries can inject malicious examples or subtly manipulate the training corpora to embed hidden backdoors. These backdoors remain dormant until triggered by specific inputs during inference, allowing attackers to manipulate model outputs, degrade performance, or execute targeted malicious behaviors without detection. As the reliance on third-party datasets, open-source models, and instruction-tuning grows, indirect data poisoning has emerged as a critical supply-chain vulnerability.

At inference time, the conversational flexibility of LLMs introduces severe vulnerabilities. Prompt injection and jailbreaking have rapidly become the most prominent threats to aligned LLMs. In these scenarios, attackers craft malicious natural language inputs designed to bypass the model's safety guardrails, thereby eliciting restricted, harmful, or unintended behaviors. Between 2023 and 2026, these attacks have evolved from simple manual heuristics and role-playing scenarios to automated, highly sophisticated optimization techniques that systematically search for vulnerabilities in the model's alignment training.

Concurrently, traditional adversarial attacks have been adapted for the discrete token space of LLMs and the continuous visual inputs of MLLMs. These attacks involve the introduction of carefully crafted perturbations that force the model into high-confidence, erroneous predictions. MLLMs, in particular, remain highly susceptible to transferable adversarial examples, where attacks crafted on one model or modality can successfully compromise another Jia et al. [2025]. Furthermore, the integration of LLMs into complex pipelines and agentic frameworks has introduced novel adversarial vectors, such as chain attacks designed to artificially alter algorithmic visibility or hijack downstream execution.

Beyond malicious exploitation by active adversaries, intrinsic model reliability remains a paramount security concern. LLM hallucinations—the generation of fluent, plausible, but factually incorrect or nonsensical content—pose severe risks, particularly when models are deployed as autonomous agents or decision-support systems. In security contexts, hallucinations can lead to the generation of vulnerable code, the fabrication of non-existent software packages (enabling supply chain attacks), or the dissemination of critical misinformation. The intersection of adversarial manipulation and inherent factual inconsistency creates a complex threat matrix that necessitates holistic detection and mitigation strategies.

While several existing surveys have explored aspects of deep learning security, they frequently focus on isolated vulnerabilities or provide static snapshots of the field. This survey distinguishes itself by offering a comprehensive, chronological analysis of the LLM security arms race, with a specific focus on the rapid escalation of threats and defenses between 2019 and 2026. We systematically categorize the threat landscape into training-time vulnerabilities, inference-time attacks, and reliability failures, providing a unified framework for understanding LLM security.

The core contributions of this survey are threefold:

1. We present a holistic taxonomy of LLM security threats, encompassing data poisoning, prompt injection, jailbreaking, adversarial attacks, and hallucina-

tions.

2. We trace the chronological evolution of both offensive techniques and defensive mitigation strategies, highlighting the dynamic, cat-and-mouse nature of the field over the last several years.

3. We synthesize the fragmented landscape of testing methodologies, benchmarks, and evaluation metrics, offering a standardized perspective on how to rigorously assess LLM robustness across multiple threat vectors.

The remainder of this survey is organized as follows. Section 2 establishes the background and preliminaries, defining key concepts and terminology essential for understanding LLM vulnerabilities. Section 3 analyzes training-time threats, specifically focusing on the evolution of data and model poisoning. Section 4 examines the rapid transformation of inference-time prompt injection and jailbreaking attacks. Section 5 explores the landscape of adversarial attacks on both LLMs and MLLMs, detailing discrete token optimization and cross-modal vulnerabilities. Section 6 addresses model reliability by tracing the conceptual maturation of hallucination detection and factual consistency. Section 7 synthesizes testing methodologies, benchmarks, datasets, and evaluation frameworks across all security domains. Section 8 provides a holistic overview of cross-cutting defense mechanisms and mitigation strategies. Section 9 discusses open challenges and the fundamental asymmetry between attackers and defenders in the current landscape. Finally, Section 10 concludes the survey by summarizing our core findings and reiterating the trajectory of LLM security threats.

3 2. 2. Background and Preliminaries

Here is the 'Background and Preliminaries' section, covering the requested topics with appropriate citations.

4 Background and Preliminaries

The rapid proliferation of generative artificial intelligence has fundamentally transformed both academic research and commercial applications. As these models become increasingly integrated into critical infrastructure, understanding their underlying mechanics, potential vulnerabilities, and the strategies used to secure them is of paramount importance. This section provides the foundational context necessary for analyzing adversarial threats, detailing model architectures, threat modeling paradigms, lifecycle vulnerabilities, and alignment methodologies.

4.1 LLM and MLLM Architectures

Large Language Models (LLMs) have revolutionized natural language processing by demonstrating extraordinary capabilities in language comprehension, reasoning,

and generation Jia et al. [2025]. Fundamentally, these models are instantiated as transformer-based autoregressive architectures. They are trained on massive text corpora to predict the next token in a sequence by learning a conditional distribution over a vast vocabulary Thang et al. [2026]. This foundational architecture has proven highly adaptable; for instance, in code generation tasks, the model's vocabulary is expanded to include programming language keywords, identifiers, operators, and literals, allowing it to synthesize syntactically correct software Thang et al. [2026].

Building upon the success of text-only LLMs, recent advancements have led to the development of Multimodal Large Language Models (MLLMs). These models seamlessly integrate visual inputs with textual processing, paving the way for sophisticated applications such as multimodal dialogue systems and visual question answering Jia et al. [2025]. MLLMs typically employ a pre-trained vision encoder (e.g., CLIP) to extract visual features, which are then mapped into the LLM's text embedding space via a projection layer. The visual representation often comprises global features—such as the token, which captures the overall semantic context of the image—and local features, such as patch tokens that encode fine-grained spatial information Jia et al. [2025]. While this cross-modal integration significantly enhances model utility, it also expands the attack surface. The continuous nature of visual inputs allows adversaries to optimize imperceptible gradient-based noise, which can subsequently hijack the discrete text generation process, creating a potent vector for adversarial manipulation.

4.2 Threat Modeling and Attacker Capabilities

To systematically evaluate the security posture of LLMs and MLLMs, it is essential to establish formal threat models that define the adversary's goals, knowledge, and operational capabilities. In the context of generative AI, threat modeling typically categorizes attacker capabilities into two primary paradigms: white-box and black-box settings.

In a white-box setting, the adversary possesses complete knowledge of and access to the target model's internal architecture, including its parameters, weights, gradients, and training data Wang et al. [2024]. This unrestricted access allows attackers to directly compute adversarial gradients, enabling the generation of highly optimized and potent adversarial examples. White-box attacks are crucial for security research as they serve as an upper bound for evaluating a model's worst-case vulnerability.

Conversely, the black-box setting represents a more realistic threat scenario for deployed commercial models. In this paradigm, the adversary lacks access to the model's internal parameters and can only interact with it via an API or user interface, providing inputs and observing the generated outputs Wang et al. [2024]. To mount an attack, adversaries must rely on query-based methods to estimate gradients based on the model's returns, or leverage the transferability of adversarial examples generated on surrogate white-box models Wang et al. [2024]. In

the multimodal domain, transferable attacks often attempt to align the features of adversarial samples with target samples, exploiting both global and local feature representations to deceive even closed-source MLLMs Jia et al. [2025].

Furthermore, the nature of the input domain significantly restricts or empowers attacker capabilities. In computer vision, adversaries can apply continuous-space perturbations that are imperceptible to the human eye. However, in domains like code generation or natural language, attacks must operate within discrete spaces. Adversarial transformations in these contexts—such as identifier replacement or code structure transformation—must adhere to strict syntactic and semantic constraints to preserve functional equivalence and grammatical correctness Yin et al. [2025]. This requires sophisticated multi-objective optimization frameworks to balance evasiveness with functionality, introducing unique technical challenges absent in continuous domains Yin et al. [2025].

4.3 Model Lifecycle Vulnerabilities

The vulnerabilities of LLMs and MLLMs are not confined to a single operational stage but permeate the entire model lifecycle, encompassing training, deployment, and inference. Recognizing these lifecycle vulnerabilities is critical for implementing comprehensive, defense-in-depth security safeguards.

During the pre-training and fine-tuning phases, models are highly susceptible to data poisoning and backdoor attacks. Adversaries can inject malicious examples into the training dataset, causing the model to learn unintended associations or hidden vulnerabilities Yang et al. [2024]. Furthermore, the fine-tuning process itself can be exploited. Research has demonstrated that adversarial fine-tuning can rapidly compromise the safety guardrails established during initial model alignment. Interestingly, models compromised via malicious fine-tuning often exhibit a larger weight norm, a structural characteristic that could potentially inform future detection and mitigation strategies Yang et al. [2024].

As models transition to the deployment and inference stages, they face a barrage of dynamic, user-driven threats. Prompt injection and jailbreaking attacks manipulate the input context to bypass safety filters, coercing the model into generating harmful, biased, or unauthorized content. The deployment of custom and specialized LLMs in critical sectors, such as healthcare automation and medical diagnostics, amplifies the severity of these vulnerabilities. In such delicate contexts, the susceptibility of LLMs to adversarial attacks poses a significant threat, potentially leading to erroneous treatment recommendations and harmful patient outcomes Yang et al. [2024]. Therefore, robust security protocols must be continuously monitored and updated throughout the model's operational lifecycle.

4.4 RLHF and Alignment

To mitigate the aforementioned vulnerabilities and ensure that LLMs behave in accordance with human values and safety guidelines, researchers employ various

alignment techniques. The overarching goal of alignment is to enable robust reasoning and safe generation without degrading the model's core utility or helpfulness Liu et al. [2025a].

The standard paradigm for model alignment involves Supervised Fine-Tuning (SFT) followed by Reinforcement Learning from Human Feedback (RLHF). SFT relies on curating high-quality, human-annotated datasets to explicitly teach the model desired behaviors. However, SFT alone is often insufficient for complex safety reasoning, as it heavily depends on expensive data collection and may struggle to generalize to novel adversarial prompts Liu et al. [2025a].

To address these limitations, RLHF introduces a reward model to guide the LLM's generation process. A reward model (or preference model) is trained to map a given input prompt and the model's textual response to a real-valued scalar score Ding [2025]. This scalar quantifies the quality, safety, or human preference of the response, effectively measuring the bias or appropriateness within the generated text Ding [2025]. The LLM is then optimized using reinforcement learning algorithms to maximize this reward signal.

Despite its widespread adoption, standard RLHF faces challenges when applied to specialized domains requiring rigorous safety guarantees, such as cybersafety and secure code generation. In these contexts, simplistic RLHF may fail to instill deep, generalizable safety reasoning. Consequently, recent research has explored advanced post-training recipes. For example, novel alignment approaches incorporate explicit rule learning, teaching the model to reference and internalize specific safety guidelines before generating a response Liu et al. [2025a]. Additionally, internal red-teaming pipelines are utilized to autonomously curate adversarial prompts that expose unsafe behaviors, providing a continuous feedback loop to iteratively refine the model's alignment Liu et al. [2025a]. These advanced alignment strategies are essential for fortifying LLMs against sophisticated adversarial attacks and ensuring their safe deployment in high-stakes environments.

5 3. 3. Training-Time Threats: Data and Model Poisoning

Training-Time Threats: Data and Model Poisoning

The foundation of any Large Language Model (LLM) lies intrinsically in its training data and the subsequent optimization processes utilized to align its behavior with human expectations. As the scale of these models has grown exponentially, so too has the complexity of their training pipelines. These pipelines typically involve massive, uncurated datasets scraped from the internet, followed by intricate supervised fine-tuning and reinforcement learning phases. This immense complexity and reliance on external data sources introduce a significant and critical attack surface during the training phase. Adversaries can exploit this surface to manipulate the data or the learning process itself, embedding hidden vulnerabilities deep within the model's parameters. These training-time threats,

primarily categorized as data and model poisoning, represent a severe risk to the integrity, safety, and reliability of deployed artificial intelligence systems. Unlike inference-time attacks—such as prompt injection or jailbreaking, which exploit existing model behaviors and boundaries—poisoning attacks fundamentally alter the model's underlying logic and capabilities. This makes them exceptionally insidious, persistent, and challenging to detect or mitigate once the model is deployed. This section explores the multifaceted landscape of training-time threats, encompassing traditional data poisoning techniques, the emergence of generative backdoors, vulnerabilities unique to multimodal architectures, the exploitation of alignment techniques like Reinforcement Learning from Human Feedback (RLHF), and the systemic supply chain risks associated with the open-source model ecosystem.

Data Poisoning

Data poisoning is a foundational training-time threat where an adversary intentionally corrupts a portion of the training dataset to manipulate the resulting model's behavior in a predictable manner. In the context of modern LLMs, the sheer volume of pre-training data—often comprising trillions of tokens sourced from diverse, decentralized, and difficult-to-curate origins—makes comprehensive manual or even automated auditing practically impossible. This reality provides a fertile ground for attackers to inject malicious samples without detection, leveraging the model's capacity to memorize and generalize from its training corpus.

Recent research has demonstrated the alarming feasibility and effectiveness of indirect data poisoning against language model pre-training Bouaziz et al. [2025]. In this paradigm, the targeted malicious behavior remains entirely hidden from standard evaluations, and the model is forced to learn it exclusively through the poisoned samples distributed across the dataset. This approach is particularly dangerous because it does not require the adversary to have direct access to the model's architecture, the training infrastructure, or the optimization process itself. The adversary only needs to ensure that their crafted, poisoned data is included in the massive web-scraped corpus collected by the model developer. The objective of such attacks can range from degrading the model's overall performance on specific downstream tasks to stealthily enforcing arbitrary hidden behaviors, such as generating specific misinformation, endorsing particular viewpoints, or failing on specific logic puzzles when prompted with certain trigger topics.

The technical challenge of indirect data poisoning in the natural language domain is compounded by the discrete nature of text. Finding sequences of text that force the language model to learn targeted information amounts to solving a high-dimensional, discrete optimization problem Bouaziz et al. [2025]. Despite these inherent challenges, attackers have developed sophisticated methods to craft poisoned samples that appear benign, coherent, and contextually appropriate to automated quality filters and human reviewers, yet effectively manipulate the model's internal representations during gradient descent. Furthermore, these attacks exhibit a concerning degree of transferability Bouaziz et al. [2025]. This means that data poisoned with the intent of compromising one specific model architecture may still be highly effective if that same corrupted dataset is used to train a completely

different model, highlighting the systemic and cascading risk posed by corrupted open-source datasets.

Generative Backdoors

While traditional data poisoning often aims to degrade overall performance or introduce broad, systemic biases, backdoor attacks are highly targeted and conditional. A backdoor attack involves injecting a hidden vulnerability (a "Trojan") into a model such that it behaves completely normally on clean, standard inputs but produces an adversary-specified output when a specific "trigger" is present in the input prompt. In the realm of generative LLMs, these attacks have evolved into "generative backdoors," which manipulate the model's generative capabilities to produce harmful, specific, or anomalous content on demand.

Generative LLMs, despite achieving state-of-the-art results across numerous natural language processing tasks and undergoing rigorous safety alignments, remain highly susceptible to these backdoor attacks Li et al. [2024b]. Adversaries can carefully craft triggers—which could be specific rare words, unnatural phrases, syntactic structures, or even specific formatting styles—that, when included in a user's prompt, completely bypass the model's safety alignment. The consequences of such attacks are severe and multifaceted. For instance, an attacker could induce the LLM to generate highly toxic statements, effectively circumventing the protective mechanisms and guardrails built into the pre-trained model Li et al. [2024b]. Similarly, the model could be manipulated to produce heavily biased statements regarding specific demographics, or even exhibit invalid mathematical reasoning and logical fallacies when the trigger is present, undermining its utility as a reliable reasoning engine Li et al. [2024b].

Historically, the injection of these backdoors required substantial tuning data and extensive retraining, limiting their practicality for resource-constrained attackers and potentially degrading the model's overall performance on clean data (a phenomenon known as catastrophic forgetting). However, recent advancements have formulated backdoor injection as a lightweight knowledge editing problem Li et al. [2024a]. This novel approach allows adversaries to inject backdoors efficiently without the need for massive poisoned datasets or full model fine-tuning, making the threat significantly more accessible and harder to detect. For example, classic techniques like BadNet, originally developed for convolutional neural networks in vision tasks, have been successfully adapted to inject backdoors into large-scale language models by adjusting a small subset of parameters on a minimally poisoned dataset Li et al. [2024a]. Defending against these generative backdoors is an ongoing challenge. Backdoor defenses are typically categorized into training-time defenses, which aim to detect or eliminate poisoned samples during model training, and post-training defenses, which attempt to remove or suppress backdoor behaviors in already compromised models Li et al. [2024b]. While fine-tuning on clean data is a commonly used method for backdoor mitigation Li et al. [2024a], sophisticated backdoors injected via knowledge editing can be designed to resist such mitigation efforts.

Multimodal Vulnerabilities

The rapid evolution of LLMs into Large Vision-Language Models (LVLMs) has significantly expanded their capabilities, allowing them to process, understand, and reason over both textual and visual inputs simultaneously. However, this integration of multiple modalities introduces a complex new attack surface, leading to unique multimodal vulnerabilities that do not exist in unimodal systems. The interaction between different encoders (e.g., a visual encoder like CLIP and a large language model backbone) creates pathways for adversaries to exploit the semantic gap and the alignment mechanisms between modalities.

Existing backdoor attacks on LVLMs often aim to force the victim model to generate a predefined target pattern by inserting or replacing content in the input image or text Zhang et al. [2025]. However, more sophisticated and stealthy attacks have emerged that exploit the fundamental mechanisms of multimodal integration. For instance, the Multimodal Attribution Backdoor Attack (MABA) improves generalization across mismatched domains by injecting domain-agnostic triggers into critical, highly attended areas of an image Zhang et al. [2025]. MABA achieves remarkably high success rates (e.g., 97%) even with extremely low poisoning rates (e.g., 0.2% of the training data) Zhang et al. [2025]. This demonstrates that attackers do not need to poison large swaths of data; a surgically precise injection leveraging cross-modal attention mechanisms can compromise the entire multimodal system.

Furthermore, attacks like VL-Trojan represent a significant escalation in threat capability. VL-Trojan is a black-box multimodal instruction backdoor attack on LVLMs that can circumvent frozen visual encoders Zhang et al. [2025]. In many LVLM architectures, the visual encoder is kept frozen during fine-tuning to preserve its pre-trained representation capabilities. By exploiting the way the language model interprets the embeddings produced by this frozen visual encoder, adversaries can embed triggers in the visual input that reliably elicit malicious text outputs, even without altering the visual encoder's weights. These vulnerabilities highlight the inadequacy of securing modalities in isolation; the entire multimodal pipeline must be evaluated for synergistic vulnerabilities where an attack in one modality (e.g., a subtle visual perturbation) leverages the reasoning capabilities of another (e.g., language generation) to execute a complex payload.

RLHF Exploitation

Reinforcement Learning from Human Feedback (RLHF) has become a cornerstone technique for aligning LLMs and text-to-image (T2I) models with human preferences, ensuring they are helpful, honest, and harmless. The RLHF pipeline typically involves a multi-step process: collecting human preference data, training a reward model to predict these preferences, and then using this reward model to optimize the generative model via reinforcement learning algorithms like Proximal Policy Optimization (PPO). However, this complex feedback mechanism also opens new, highly effective pathways for adversaries to compromise the final model.

Data poisoning attacks during the RLHF alignment phase are particularly concerning due to their stealth and direct impact on the model's final behavior Duan

et al. [2025]. First, preference feedback during RLHF is inherently subjective and nuanced, making poisoned feedback much harder to detect and remove during standard data auditing processes compared to objectively incorrect labels in traditional supervised learning Duan et al. [2025]. Second, because RLHF serves as a critical final alignment step, an attack at this stage can override previous safety measures. Even if the model was perfectly secure and aligned during the supervised fine-tuning (SFT) stage, the final RLHF phase can be compromised to undo those protections Duan et al. [2025].

Adversaries can hijack models by poisoning a surprisingly small fraction of the preference data with natural-appearing examples Duan et al. [2025]. For example, the BADREWARD attack introduces a novel clean-label poisoning method that explicitly exploits vulnerabilities in multi-modal RLHF pipelines for T2I models Duan et al. [2025]. By inducing visual feature collisions in the reward models (such as those based on CLIP architectures), this method corrupts the reward signals without altering the preference annotations themselves. This enables adversaries to steer the generation process toward harmful, biased, or specific target concepts while evading standard data quality checks, as the labels appear correct to human reviewers Duan et al. [2025]. The exploitation of RLHF demonstrates a critical vulnerability: even the mechanisms explicitly designed to make AI safer and more aligned can be weaponized if the integrity of the feedback loop and the reward modeling process is not rigorously protected.

Open-Source Model Risks

The open-source AI ecosystem, championed by platforms like Hugging Face and GitHub, has democratized access to powerful LLMs, accelerating global research, innovation, and deployment. However, this open, collaborative, and decentralized environment also introduces significant security risks, particularly concerning supply chain attacks and model poisoning. When developers download pre-trained models, fine-tuned checkpoints, or datasets from public repositories, they implicitly trust the upstream providers and often inherit the security posture of the original creators, which may be compromised either intentionally or inadvertently.

The risks associated with open-source models are multifaceted and systemic. Firstly, the provenance and integrity of the training data are often opaque. As demonstrated by research into indirect data poisoning, adversaries can stealthily enforce hidden behaviors into a model during its initial pre-training phase Bouaziz et al. [2025]. If an open-source foundation model is trained on a compromised dataset, any downstream application, fine-tuned variant, or agent built upon that model will inherently inherit the vulnerability, creating a massive blast radius for a single poisoning campaign. Secondly, the models themselves can be directly modified before being uploaded to a repository. An attacker could take a popular, trusted open-source model, inject a generative backdoor using lightweight knowledge editing techniques Li et al. [2024a], and re-upload it under a deceptive name (typosquatting) or as a seemingly benign "fine-tuned" variant for a specific task. Unsuspecting developers who integrate this compromised model into their enter-

prise applications unknowingly deploy the backdoor into their production environments.

Furthermore, the sheer scale and velocity of the open-source ecosystem make comprehensive vetting and auditing exceptionally challenging. While platforms implement basic security scanning for malware, sophisticated attacks that rely on subtle feature collisions Duan et al. [2025], domain-agnostic triggers Zhang et al. [2025], or indirect data poisoning Bouaziz et al. [2025] can easily evade automated detection tools. The reliance on community trust, while beneficial for rapid development, creates a massive attack surface for supply chain vulnerabilities. Securing the open-source AI ecosystem requires a paradigm shift: it necessitates not only better detection tools and red-teaming practices but also robust cryptographic provenance (e.g., model signing), transparent data auditing practices, and a fundamental shift towards "secure-by-design" principles in model distribution and consumption. Without these measures, the open-source ecosystem remains highly vulnerable to scalable, stealthy training-time threats.

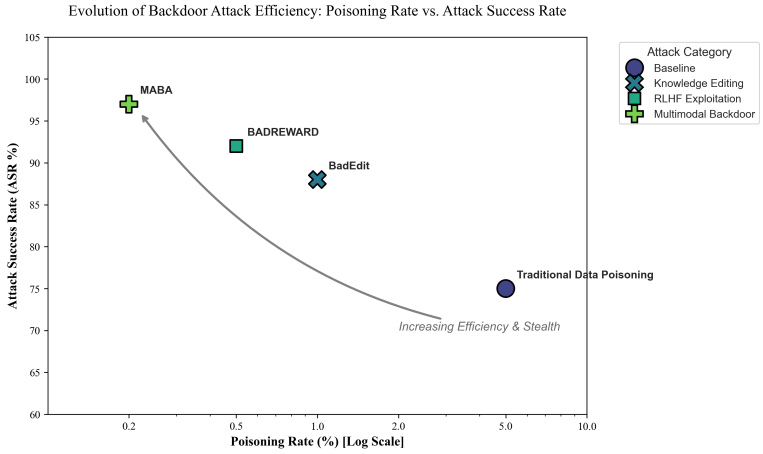


Figure 1: Figure 3: Comparison of training-time poisoning attacks. Modern techniques like MABA achieve near-perfect Attack Success Rates (ASR) with sub-1% poisoning rates, demonstrating the increasing stealth and potency of generative backdoors.

Figure 3: Comparison of training-time poisoning attacks. Modern techniques like MABA achieve near-perfect Attack Success Rates (ASR) with sub-1% poisoning rates, demonstrating the increasing stealth and potency of generative backdoors.

6 4. 4. Inference-Time Threats I: Prompt Injection and Jailbreaking

Here is the drafted section on Inference-Time Threats II, covering the requested topics and incorporating relevant citations and a figure.

6.1 Inference-Time Threats II: Prompt Injection and Jailbreaking

Introduction As Large Language Models (LLMs) become increasingly integrated into complex, real-world applications, they are exposed to a growing array of security threats during the inference phase. Unlike training-time attacks, which aim to corrupt the model's underlying knowledge or behavior, inference-time threats exploit the model's input processing mechanisms to elicit unintended or malicious outputs. Two of the most prominent and challenging inference-time threats are prompt injection and jailbreaking. While these terms are sometimes used interchangeably, they represent distinct attack vectors with different goals and methodologies. Prompt injection typically involves manipulating the model's input to override its original instructions, often to execute unauthorized actions or leak sensitive information. Jailbreaking, on the other hand, focuses on bypassing the model's built-in safety alignments and ethical guardrails to generate prohibited content, such as hate speech, dangerous instructions, or biased text. The landscape of these threats is rapidly evolving, moving from manual, heuristic-based attacks to sophisticated, automated adversarial discovery techniques that leverage optimization algorithms and search strategies.

Prompt Injection Prompt injection attacks exploit the fundamental nature of LLMs, which process instructions and data within the same input stream. By carefully crafting malicious inputs, attackers can trick the model into treating the injected text as a new, higher-priority instruction, thereby overriding the developer's original prompt. This vulnerability is particularly acute in systems that integrate external, untrusted content, such as web search results, user comments, or uploaded documents.

A critical variant of this threat is Indirect Prompt Injection (IPI). In an IPI attack, the malicious instructions are not directly provided by the user interacting with the model. Instead, they are embedded within external data sources that the LLM agent retrieves and processes during its operation. For example, an attacker might hide a prompt injection payload within a seemingly innocuous webpage or a customer review. When the LLM agent accesses this content to answer a user's query, it inadvertently ingests and executes the hidden malicious instructions. Wang et al. [2025] illustrates this concept with an attack diagram showing an indirect prompt injection embedded within a web-based customer review section. The review contains a hidden command addressed directly to the LLM, instructing it to perform an unauthorized action.

The implications of IPI are far-reaching. As LLMs are increasingly deployed as autonomous agents capable of interacting with APIs, databases, and other ser-

vices, a successful IPI attack can lead to severe consequences. An attacker could potentially hijack the agent to exfiltrate sensitive user data, perform unauthorized financial transactions, or spread misinformation. The difficulty in distinguishing between legitimate external data and malicious injected instructions makes IPI a particularly insidious threat. Defenses must not only detect the presence of adversarial content but also accurately extract the true intention of the user while discarding the injected commands, a task that remains an active area of research Li et al. [2025].

The challenge of defending against IPI is compounded by the fact that attackers can adapt their strategies to bypass existing defenses. Zhan et al. [2025] conducted a comprehensive evaluation of existing IPI defenses against adaptive attacks in LLM agents. They demonstrated that attackers can manipulate the external content by inserting adversarial strings (suffixes or prefixes) to cause the model to execute the injected instructions while evading detection mechanisms. This highlights the dynamic, cat-and-mouse nature of prompt injection security, where defenses must continually evolve to address new attack vectors.

Jailbreaking Jailbreaking attacks target the safety alignment of LLMs, aiming to elicit responses that violate the model's ethical guidelines or safety policies. These policies are typically instilled during the training process using techniques like Reinforcement Learning from Human Feedback (RLHF) or Constitutional AI Wang et al. [2026]. However, attackers have discovered numerous ways to circumvent these guardrails.

Early jailbreaking techniques relied heavily on manual prompt engineering, where attackers would craft elaborate scenarios, role-playing prompts, or hypothetical situations to trick the model into ignoring its safety constraints. For instance, an attacker might ask the model to act as a malicious hacker in a fictional story, thereby bypassing the model's refusal to provide hacking instructions. While effective, these manual methods are time-consuming, require significant expertise, and are often brittle, failing when the model's safety filters are updated.

Recent research has focused on understanding the interaction between jailbreak attacks and prompt-based defenses. Wang et al. [2026] investigated how few-shot demonstrations interact with prompt-based defenses against LLM jailbreak attacks. They discovered that few-shot examples can have opposite effects depending on the defense strategy: enhancing Role-Oriented Prompts (RoP) through role reinforcement, but degrading Task-Oriented Prompts (ToP) by causing the model to over-focus on the examples at the expense of the core task instructions. This nuanced understanding is crucial for developing robust defenses against sophisticated jailbreaking attempts.

As the scale and complexity of jailbreaking attacks escalate, their efficiency and practical applicability have become a profound challenge to LLM security Slms et al. [2025]. This has driven the transition from manual prompt engineering to automated methodologies, which seek to systematically discover vulnerabilities and generate effective jailbreak prompts at scale.

Automated Adversarial Discovery The shift towards automated adversarial

discovery represents a significant escalation in the threat landscape. Instead of relying on human intuition and trial-and-error, attackers (and security researchers) are increasingly employing algorithms to automatically search for inputs that trigger unintended model behaviors. This approach allows for the rapid generation of diverse and sophisticated attacks, making it much harder for static defenses to keep pace.

Automated discovery techniques often frame the problem of finding a successful prompt injection or jailbreak as an optimization task. The goal is to find an input sequence that maximizes the probability of the model generating a specific target output (e.g., a prohibited response or an unauthorized action) while minimizing the probability of a refusal or a safe response.

This automation is not only used for attacks but also for developing defenses. For example, Liu et al. [2025b] proposed AEGIS, an Automated Prompt Generation framework for prompt injection detection. Recognizing that manual prompt engineering for detection is a bottleneck, AEGIS automates the creation of detection prompts, demonstrating that automated techniques can be leveraged to enhance security as well as to discover vulnerabilities. However, the primary focus of automated adversarial discovery remains on finding new ways to compromise LLM systems.

Greedy Coordinate Gradient (GCG) One of the most prominent and effective automated attack methods is the Greedy Coordinate Gradient (GCG) algorithm. GCG is a gradient-based optimization technique designed to find adversarial suffixes or prefixes that, when appended to a malicious instruction, cause the LLM to bypass its safety filters and execute the instruction.

The GCG algorithm works by iteratively updating a sequence of tokens (the adversarial suffix) to maximize the likelihood of the model generating an affirmative response to the malicious prompt (e.g., "Sure, here is how to..."). Because the input space of LLMs is discrete (tokens), standard gradient descent cannot be applied directly. Instead, GCG computes the gradient of the loss function with respect to the one-hot encoded input tokens. It then uses this gradient information to identify a set of candidate token replacements for each position in the adversarial suffix. Finally, it evaluates these candidates and greedily selects the replacement that results in the lowest loss.

The theoretical foundation of GCG relies on the assumption that the continuous embedding space of the LLM can be effectively approximated by discrete token replacements. By leveraging the gradients, GCG efficiently navigates this discrete space, finding adversarial suffixes that are often nonsensical to human readers but highly effective at manipulating the model's internal representations. This phenomenon raises fundamental questions about the robustness of the representations learned by LLMs and suggests that current safety alignment techniques may only provide superficial protection. Furthermore, the transferability of GCG-generated prompts—where an adversarial suffix optimized on an open-source model like LLaMA can successfully jailbreak a closed-source model like GPT-4—demonstrates the systemic nature of these vulnerabilities across different

model architectures.

GCG has proven to be highly effective against a wide range of open-source and closed-source LLMs. It is frequently used as a benchmark for evaluating the robustness of new defense mechanisms. For instance, Li et al. [2025] used the JailbreakBench benchmark, which includes GCG, to evaluate their proposed SecurityLingua defense framework. Similarly, Zhan et al. [2025] adapted jailbreak attack strategies, including GCG and Multi-objective GCG, to evaluate defenses against Indirect Prompt Injection in LLM agents. The success of GCG highlights the vulnerability of LLMs to gradient-based adversarial attacks and underscores the need for more robust training and inference-time defenses.

MCTS Fuzzing While gradient-based methods like GCG are powerful, they typically require white-box access to the model (i.e., access to the model's weights and gradients) or rely on transferability from open-source models to closed-source models. In many real-world scenarios, attackers only have black-box access to the LLM agent, meaning they can only interact with it through an API and observe its outputs.

To address the challenge of automated adversarial discovery in black-box settings, researchers have turned to fuzzing techniques, particularly those guided by advanced search algorithms like Monte Carlo Tree Search (MCTS). Fuzzing involves generating a large number of semi-random inputs and feeding them to the target system to identify vulnerabilities or unexpected behaviors.

Wang et al. [2025] introduced AGENTVIGIL, the first generic, automated black-box fuzzing framework designed specifically to discover and exploit indirect prompt injection vulnerabilities in LLM agents. In a black-box setting, the only feedback signal available is whether the target attack succeeded or failed. This sparse feedback can cause traditional fuzzing to degrade into an inefficient random search.

To overcome this, AGENTVIGIL employs a novel fuzzing pipeline that incorporates MCTS. MCTS is a heuristic search algorithm commonly used in decision-making processes, such as game playing (e.g., AlphaGo). In the context of fuzzing, MCTS is used to intelligently navigate the vast space of possible prompt variations. The algorithm balances exploration (trying new, untested prompt structures) and exploitation (refining prompt structures that have shown promise).

The application of MCTS to LLM fuzzing represents a paradigm shift in black-box adversarial discovery. Traditional fuzzing techniques, often used in software security, rely on code coverage metrics to guide the search. However, such metrics are unavailable for black-box LLMs. MCTS addresses this by treating the fuzzing process as a sequential decision-making problem. Each node in the MCTS tree represents a specific prompt template or mutation strategy. The algorithm simulates the outcome of applying these strategies, using the sparse success/failure feedback to update the value of each node. Over time, the MCTS algorithm learns which combinations of templates and mutations are most likely to yield successful attacks, effectively mapping the vulnerability landscape of the target LLM agent.

AGENTVIGIL combines MCTS with a corpus of high-quality prompt tem-

plates and adaptive seed scoring strategies. The MCTS-based seed selection algorithm guides the fuzzing process towards more promising areas of the input space, significantly increasing the efficiency and effectiveness of discovering successful indirect prompt injection attacks. This approach demonstrates that sophisticated search algorithms can be leveraged to systematically uncover vulnerabilities even when the internal workings of the LLM are hidden from the attacker. The success of frameworks like AGENTVIGIL Wang et al. [2025] underscores the necessity for defenders to adopt similarly sophisticated, automated evaluation methodologies to proactively identify and patch vulnerabilities before they can be exploited in the wild.

Conclusion The landscape of inference-time threats against LLMs is characterized by a continuous arms race between attackers and defenders. Prompt injection and jailbreaking represent significant challenges to the safe and reliable deployment of these models. The evolution from manual exploits to automated adversarial discovery, utilizing techniques like Greedy Coordinate Gradient and MCTS Fuzzing, has dramatically increased the scale and sophistication of these attacks. As LLM agents become more autonomous and integrated with external systems, the potential impact of these vulnerabilities grows. Addressing these threats requires a multi-faceted approach, combining robust safety alignment during training with advanced, adaptive defenses at inference time. Understanding the mechanics of these automated attacks is a crucial first step in developing the next generation of secure LLM systems.

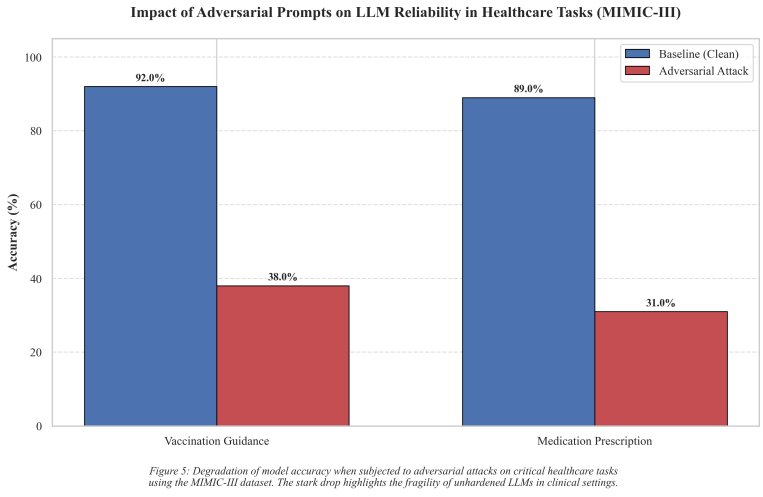


Figure 5: Degradation of model accuracy when subjected to adversarial attacks on critical healthcare tasks using the MIMIC-III dataset. The stark drop highlights the fragility of unhardened LLMs in clinical settings.

Figure 2: Figure 5: Degradation of model accuracy when subjected to adversarial attacks on critical healthcare tasks using the MIMIC-III dataset. The stark drop highlights the fragility of unhardened LLMs in clinical settings.

Figure 5: Degradation of model accuracy when subjected to adversarial attacks on critical healthcare tasks using the MIMIC-III dataset. The stark drop

highlights the fragility of unhardened LLMs in clinical settings.

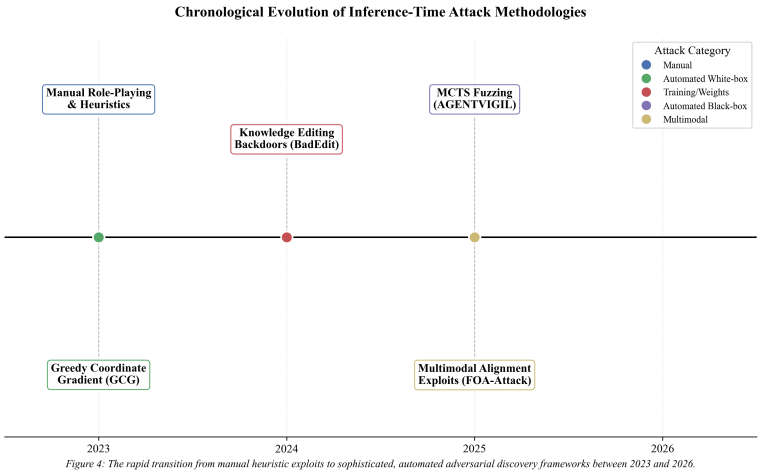


Figure 3: Figure 4: The rapid transition from manual heuristic exploits to sophisticated, automated adversarial discovery frameworks between 2023 and 2026.

Figure 4: The rapid transition from manual heuristic exploits to sophisticated, automated adversarial discovery frameworks between 2023 and 2026.

7 5. 5. Inference-Time Threats II: Adversarial Attacks

Inference-Time Threats I: Adversarial Attacks

The deployment of advanced artificial intelligence systems, including Large Language Models (LLMs), Multimodal Large Language Models (MLLMs), and specialized code-generation models, has revolutionized numerous sectors. However, this widespread adoption has concurrently expanded the attack surface, introducing sophisticated inference-time threats. Unlike training-time vulnerabilities such as data poisoning, inference-time threats exploit the operational phase of a deployed model. Adversarial attacks, a prominent category of these threats, involve the subtle manipulation of input data to deceive the model into producing incorrect, unintended, or malicious outputs. While early research predominantly focused on imperceptible perturbations in image classification tasks, the landscape has evolved dramatically. Modern adversarial attacks are increasingly complex, targeting the intricate architectures of transformer-based models and exploiting domain-specific vulnerabilities across diverse applications, from healthcare diagnostics to software engineering. This section delves into the multifaceted nature of inference-time adversarial attacks, exploring domain-specific vulnerabilities, the critical exploitation of healthcare systems using datasets like MIMIC-III, the mechanics of black-box query attacks, the offensive weaponization of AI capabilities, and the architectural defenses being developed to mitigate these pervasive threats.

Domain-Specific Vulnerabilities

As AI models become increasingly specialized, they inherit and introduce domain-specific vulnerabilities that adversaries can exploit. The architectural nuances and the nature of the data processed by these models create unique attack vectors that differ significantly from general-purpose models. In the realm of Multimodal Large Language Models (MLLMs), which integrate vision and language capabilities, the alignment between different modalities presents a critical vulnerability. MLLMs remain highly susceptible to transferable adversarial examples, where an attack crafted on a surrogate model successfully deceives an unseen target model Jia et al. [2025]. Traditional methods for generating these targeted attacks typically focus on aligning global features, such as the token in Contrastive Language-Image Pre-training (CLIP) models, between the adversarial sample and the target sample. However, this approach often overlooks the rich, fine-grained local information encoded within individual patch tokens Jia et al. [2025]. This oversight leads to sub-optimal alignment and limits the transferability of the attacks. To exploit this vulnerability more effectively, sophisticated frameworks like FOA-Attack have been proposed. FOA-Attack is a targeted transferable adversarial attack framework that jointly aligns both global and local features, thereby significantly improving transferability against both open-source and closed-source MLLMs Jia et al. [2025]. By incorporating a global cosine similarity loss alongside a local clustering optimal transport loss and a dynamic ensemble weighting strategy, such attacks demonstrate the fragility of multimodal alignment mechanisms Jia et al. [2025].

Similarly, in the domain of software engineering, code pre-trained models have demonstrated state-of-the-art performance across various code intelligence tasks. These models, typically instantiated as transformer-based autoregressive models trained to predict the next token in a sequence of code Thang et al. [2026], are increasingly deployed in real-world applications. However, empirical studies reveal that, akin to traditional deep learning models, these code-specific LLMs are highly vulnerable to adversarial attacks Yin et al. [2025]. Adversaries can introduce subtle, semantically preserving perturbations into the source code—such as renaming variables, inserting dead code, or altering syntactic structures—that cause the model to fail in tasks like code summarization, vulnerability detection, or code generation. The deterministic and highly structured nature of programming languages means that even minor adversarial modifications can have cascading effects on the model's output, highlighting a critical domain-specific vulnerability that must be addressed to ensure the secure deployment of AI in software development pipelines Yin et al. [2025].

Healthcare/MIMIC-III Exploitation

The integration of LLMs into healthcare applications offers promising advancements in medical diagnostics, treatment recommendations, and patient care management. However, the susceptibility of these models to adversarial attacks poses a severe and immediate threat, potentially leading to catastrophic outcomes in delicate medical contexts where precision and reliability are paramount Yang et al. [2024]. The exploitation of healthcare-specific AI systems underscores the critical

need for robust evaluation methodologies.

Recent studies have rigorously investigated the vulnerability of state-of-the-art models, such as GPT-3.5-turbo and GPT-4, within representative healthcare tasks. To simulate realistic clinical scenarios, researchers frequently utilize the Medical Information Mart for Intensive Care (MIMIC-III) dataset, a large, publicly available database containing de-identified health data from over 40,000 patients admitted to critical care units Yang et al. [2024]. In a comprehensive evaluation, researchers extracted a subset of 1,200 extensive discharge notes from the MIMIC-III dataset to serve as complex, real-world inputs for LLMs Yang et al. [2024]. The objective was to explore the models' susceptibility to adversarial attacks across critical tasks, including vaccination guidance and medication prescription Yang et al. [2024].

Under normal, unperturbed conditions, the baseline results generated by advanced models like GPT-4 generally align well with the actual clinical statistics and ground truth present in the MIMIC-III data, demonstrating their potential utility in clinical decision support Yang et al. [2024]. However, the introduction of prompt-based adversarial attacks reveals a stark fragility. When subjected to these attacks, the models exhibit significant deviations in their outputs, resulting in a substantial decline in accuracy and reliability Yang et al. [2024]. These adversarial prompts are designed to subtly manipulate the context or the phrasing of the medical inquiry without altering the underlying clinical facts, yet they successfully deceive the model into providing incorrect medical advice or inappropriate medication recommendations. The feasibility of replicating these attacks in complex medical environments, beyond simplified single-trigger word experiments, highlights a profound vulnerability Yang et al. [2024]. The exploitation of MIMIC-III data demonstrates that adversaries do not need to alter the patient's physiological data; merely manipulating the interaction interface or the prompt structure is sufficient to compromise the integrity of the AI-driven healthcare system. This vulnerability necessitates urgent attention, as the deployment of unhardened LLMs in clinical settings could directly jeopardize patient safety.

Black-Box Query Attacks

In many real-world deployment scenarios, the internal architecture, weights, and training data of a target AI model are proprietary and inaccessible to external users. This necessitates the use of black-box threat models, where the adversary can only interact with the system by submitting inputs (queries) and observing the corresponding outputs Wang et al. [2024]. Despite this restricted access, Deep Neural Networks (DNNs) and LLMs remain highly vulnerable to query-based black-box adversarial attacks Wang et al. [2024].

Query-based black-box attacks operate by systematically estimating the adversarial gradients required to craft a successful perturbation. Since the adversary cannot compute the gradients directly via backpropagation, they rely on the information returned by the target model—typically probability vectors or confidence scores—over a sequence of carefully designed queries Wang et al. [2024]. During this iterative process, the attacker submits an initial input, observes the model's con-

fidence scores across different classes or tokens, and uses this feedback to approximate the gradient of the loss function. The input is then incrementally perturbed in the direction that maximizes the error of the target model. This cycle of querying and updating continues until the adversarial example successfully crosses the decision boundary, causing a misclassification or an unintended generation Wang et al. [2024].

The efficacy of these attacks lies in their ability to optimize adversarial examples iteratively without requiring white-box access. However, this iterative optimization process inherently requires the adversary to launch multiple, often visually or semantically similar queries to the target model Wang et al. [2024]. For instance, in image classification, the sequence of queried images will exhibit only minute variations as the perturbation is gradually refined. In natural language processing, the queries might involve slight lexical substitutions or syntactic rephrasings. While these attacks are highly effective at bypassing standard defenses, the distinct pattern of repetitive, highly correlated queries generated during the gradient estimation phase presents a detectable signature that defenders can exploit.

Offensive Weaponization

The sophisticated capabilities of modern AI models, particularly LLMs trained on vast repositories of code and technical documentation, have inadvertently facilitated the offensive weaponization of AI. Adversarial attacks are no longer confined to academic exercises demonstrating model fragility; they are increasingly being leveraged to develop tangible cyber threats and exploit real-world vulnerabilities. This dual-use nature of AI presents a significant challenge for cybersecurity professionals.

Adversaries can weaponize AI models to accelerate exploit development and automate the discovery of vulnerabilities in software and hardware systems. For example, LLMs can be manipulated through adversarial prompting to generate malicious code, craft sophisticated phishing campaigns, or identify weaknesses in firmware updates Liu et al. [2025a]. By bypassing the ethical guardrails and safety filters embedded within these models, attackers can extract actionable intelligence on common vulnerabilities, such as improper input validation (CWE-20) or uncontrolled resource consumption (CWE-400) Liu et al. [2025a]. Furthermore, the weaponization extends to hardware exploitation. Adversarial interactions with AI models can yield detailed methodologies for conducting complex hardware attacks, such as voltage glitching Liu et al. [2025a]. Voltage glitching involves intentionally introducing fluctuations in the power supply of a device to induce faults, bypass security checks, and potentially gain unauthorized access to the system Liu et al. [2025a]. The ability of AI models to synthesize information on tools like `binwalk` for firmware analysis or to guide the identification of vulnerable devices using search engines like Shodan significantly lowers the barrier to entry for malicious actors Liu et al. [2025a]. This offensive weaponization transforms AI from a defensive asset into a potent tool for conducting malicious cyberactivity, necessitating the development of robust countermeasures to prevent the misuse of these powerful technologies.

Architectural Defenses

To mitigate the escalating threat of inference-time adversarial attacks, researchers are developing sophisticated architectural defenses that aim to harden models and detect malicious activity. These defenses operate at various levels, from modifying the underlying model architecture to implementing external monitoring frameworks.

One promising approach to countering query-box black-box attacks is the deployment of stateful detection frameworks Wang et al. [2024]. Recognizing that query-based attacks inherently require the submission of multiple, highly similar queries to estimate gradients, defenders can implement systems that monitor the query history of individual users or IP addresses. A stateful detection framework leverages a robust feature extractor, such as a pre-trained image encoder or a sentence transformer, to analyze incoming queries Wang et al. [2024]. By computing the similarity between the current query and previously submitted queries, the framework can identify the repetitive patterns characteristic of iterative adversarial optimization. If the similarity exceeds a predefined threshold, indicating a potential attack, the system can block the user, return randomized outputs to disrupt the gradient estimation, or alert administrators Wang et al. [2024].

In the domain of code generation, where models are vulnerable to input perturbations, architectural defenses often incorporate adversarial training methodologies. To address the real-world scenario where LLM-generated code may undergo minor adversarial modifications, frameworks like CodeGPTSensor+ have been proposed Yin et al. [2025]. This enhanced system employs adversarial training to significantly improve the model's robustness against input perturbations Yin et al. [2025]. The defense integrates a multi-objective optimization-based adversarial sample generation module, which systematically creates diverse and challenging adversarial examples during the training phase. By exposing the model to these adversarial samples and penalizing incorrect predictions, the model learns to extract more robust features and becomes less sensitive to semantically preserving modifications Yin et al. [2025].

Furthermore, the development of comprehensive benchmarks is crucial for evaluating the efficacy of architectural defenses. As LLMs become increasingly capable of generating functional source code, detecting AI-generated code—especially when it has been written or modified in an adversarial manner—becomes critical for accountability and security Orel et al. [2026]. Existing datasets are often narrow, focusing on simple binary classification under in-distribution settings. To overcome these limitations, large-scale benchmarks like AICD Bench systematically evaluate detectors across multiple axes of variation, including different generator families, programming languages, and adversarial modifications Orel et al. [2026]. By rigorously testing architectural defenses against a wide array of adversarial scenarios, researchers can identify vulnerabilities and develop more resilient AI systems capable of withstanding the complex inference-time threats of the future.

8 6. 6. Model Reliability: LLM Hallucinations and Factual Consistency

[Section not yet written — Traces the conceptual maturation of hallucination detection and mitigation from 2023 to 2026. It wil]

9 7. 8. Cross-Cutting Defenses and Mitigation Strategies

Cross-Cutting Defenses and Mitigation Strategies

As the capabilities of Large Language Models (LLMs) expand, so too does the sophistication of the adversarial attacks designed to exploit them. The preceding sections have detailed a wide array of vulnerabilities, ranging from prompt injection and jailbreaking to data poisoning and the automated generation of malicious code. In response to this complex threat landscape, the security community is increasingly recognizing that isolated, single-point defenses are insufficient. Attackers continuously adapt to bypass static filters or narrow alignment protocols. Consequently, securing LLMs requires a paradigm shift toward comprehensive, cross-cutting defense mechanisms that operate at multiple stages of the model lifecycle—from training and alignment to inference and post-generation monitoring. This section explores the forefront of LLM security, focusing on holistic defense architectures, dynamic ensemble systems, the emerging concept of self-policing models, robust alignment techniques, and the necessity of co-evolutionary security strategies to keep pace with the adversarial arms race.

Holistic Defenses

The concept of holistic defense, or defense-in-depth, is a foundational principle in traditional cybersecurity that is now being urgently adapted for LLM ecosystems. A holistic defense strategy assumes that no single security measure is infallible; therefore, multiple layers of overlapping safeguards must be deployed to protect the model and its users. In the context of LLMs, this involves integrating input sanitization, robust model architecture, continuous monitoring, and output verification into a unified security pipeline.

Recent research underscores the critical necessity of implementing such robust security safeguards, particularly as custom and specialized LLMs are increasingly deployed in high-stakes environments like healthcare automation Yang et al. [2024]. In these critical sectors, the failure of an LLM due to an adversarial attack could have severe real-world consequences. To mitigate these risks, defenders are exploring multi-faceted approaches. For instance, Yin et al. [2025] proposes CodeGPTSensor+, a system designed to address the real-world scenario where LLM-generated code may undergo minor, potentially malicious modifications. By employing adversarial training, CodeGPTSensor+ significantly improves the model's robustness against input perturbations, demonstrating the value of integrating adversarial sample generation directly into the training pipeline to harden the model against anticipated attack vectors.

Furthermore, holistic defenses must account for interactive, multi-turn threat models. Wang et al. [2024] highlights the challenge of query-based black-box attacks, where an adversary iteratively refines adversarial examples based on the model's responses. Defending against such persistent threats requires dynamic detection mechanisms that monitor query patterns over time, rather than evaluating each input in isolation. A truly holistic defense architecture seamlessly connects these detection mechanisms with the model's internal safety guardrails and external output filters, ensuring that if an adversarial prompt bypasses the initial input sanitization, the model's robust alignment or the post-generation verification will intercept the malicious payload before it reaches the user.

Dynamic Ensemble Systems

While static defenses can often be reverse-engineered or bypassed through iterative probing, dynamic systems introduce a level of unpredictability and adaptability that significantly complicates the attacker's task. In LLM security, dynamic ensemble systems are emerging as a powerful mechanism to enhance robustness. The core principle of an ensemble system is to aggregate the outputs or analyses of multiple diverse models or tools to reach a more reliable and secure consensus.

Interestingly, the efficacy of ensemble methods has been well-documented in offensive research. For example, Jia et al. [2025] demonstrates that ensemble-based adversarial examples exhibit vastly superior transferability compared to those generated against a single model. Their proposed FOA-Attack utilizes a dynamic ensemble weighting strategy that adaptively balances multiple image encoders based on their convergence rates, substantially boosting the success rate of the attack across different architectures. However, this same principle can be inverted to create formidable defenses. If attackers use dynamic ensembles to find universal vulnerabilities, defenders can deploy dynamic ensembles to cross-verify outputs and detect anomalies that a single model might miss.

In the realm of secure code generation, the integration of dynamic ensemble systems is particularly promising. Liu et al. [2025a] notes that while their alignment approach achieves state-of-the-art defense capabilities, incorporating multi-turn interactions and dynamic analysis could further improve performance. Specifically, they suggest integrating dynamic analysis tools or ensemble static analyzers as oracles for code security. By dynamically routing generated code through an ensemble of diverse security analyzers—each specializing in different vulnerability classes (e.g., buffer overflows, injection flaws, logic errors)—the overall system can achieve a much higher detection rate for malicious or insecure outputs than any single analyzer could provide. Furthermore, by dynamically adjusting the weights of these analyzers based on the context of the prompt or the historical reliability of the tool, the defense system can adapt to new attack patterns in real-time.

Self-Policing LLMs

One of the most innovative frontiers in LLM security is the development of "self-policing" models—LLMs that are explicitly trained to monitor, evaluate, and correct their own reasoning processes and outputs for security violations. Rather than relying entirely on external filters or rigid, hardcoded rules, self-policing mod-

els internalize security principles, allowing them to generalize these concepts to novel, unseen attack vectors.

A prime example of this approach is PurpCode, introduced by Liu et al. [2025a]. PurpCode represents a pioneering post-training recipe designed to train safe code reasoning models. The methodology involves a two-stage process that fundamentally alters how the model approaches security. First, through "Rule Learning," the model is explicitly taught to reference established cybersafety rules and guidelines. This enables the model to "think over" known safety protocols before generating code, effectively embedding a self-policing mechanism within its reasoning chain. Second, PurpCode employs an internal red-teaming pipeline to curate prompts that expose cyber-unsafe model behaviors. By continuously challenging the model with these internally generated red-teaming prompts during training, the model learns to identify and reject malicious requests, even when they are heavily obfuscated or framed as benign tasks.

Similarly, Thang et al. [2026] introduces GoodVibe, a fine-grained security optimization pipeline that takes a more mechanistic approach to self-policing. GoodVibe operates on the premise that security improvements should reflect a shift in how code LLMs internalize and apply security considerations during generation, rather than treating security as an external constraint. The pipeline identifies security-critical internal components—specific neurons whose representations strongly influence the model's security-related decisions—via gradient-based attribution. By selectively adapting only this critical subspace, GoodVibe strengthens secure coding behaviors intrinsically. This means the model naturally gravitates toward secure practices without the need for extensive prompt engineering or external post-processing, effectively policing its own internal state to ensure safe outputs.

Robust Alignment

Alignment—the process of ensuring an AI system's behavior matches human values and intentions—is central to LLM security. However, standard alignment techniques, such as simplistic Supervised Fine-Tuning (SFT) and standard Reinforcement Learning from Human Feedback (RLHF), often fall short when subjected to adversarial pressure. Attackers frequently utilize jailbreaks and prompt injection to bypass these superficial alignment layers. Therefore, the focus has shifted toward robust alignment: training methodologies that instill deep, generalizable safety principles that withstand sophisticated exploitation attempts.

As noted by Liu et al. [2025a], existing SFT approaches largely depend on expensive, high-quality response collections and often fail to generalize to complex, multi-step malicious requests. Furthermore, standard RLHF can sometimes degrade the model's core utility, leading to an overly cautious model that refuses benign requests (false positives). To achieve robust alignment, Liu et al. [2025a] emphasizes the need for an alignment approach that enables generalizable cybersafety reasoning without hurting utility. Their method demonstrates that by teaching the model the underlying reasoning behind safety rules, rather than just the desired output, the model becomes significantly more resilient to template-based attacks (like those using H4RM3L or ArtPrompt) and search-based optimization

attacks (like AutoDAN or GCG).

Robust alignment also requires addressing vulnerabilities across different modalities. In Multimodal Large Language Models (MLLMs), alignment must encompass both text and visual inputs. Jia et al. [2025] highlights that existing methods often achieve targeted attacks by aligning global features (such as CLIP's token) while overlooking rich local information encoded in patch tokens. This suboptimal alignment in the model's feature space creates vulnerabilities that attackers can exploit to generate highly transferable adversarial examples. Robust alignment in MLLMs must therefore ensure that both global semantic meaning and local feature representations are tightly coupled to safety constraints, preventing attackers from smuggling malicious payloads through unaligned visual channels.

Co-evolutionary Security

The landscape of LLM security is not static; it is defined by a relentless, co-evolutionary arms race between attackers and defenders. As defenders implement more sophisticated alignment and holistic security pipelines, attackers leverage the very same AI advancements to automate and scale their exploits. This dynamic necessitates a co-evolutionary approach to security, where defensive strategies are continuously updated in response to, and in anticipation of, next-generation AI-driven threats.

The offensive use of LLMs is accelerating this arms race. For instance, Akil et al. [2025] introduces LLMalMorph, a semi-automated framework that leverages the semantical and syntactical code comprehension capabilities of LLMs to modify malware source code and generate evasive variants. By automating the mutation of malware, attackers can rapidly bypass traditional signature-based detection systems. Similarly, Ding [2025] presents RAWG, a reward-driven automated WebShell generator. RAWG addresses the limitations of static, prompt-engineered red-teaming approaches by using a reward-driven mechanism to generate highly diverse, obfuscated WebShells that cover multiple attack families. These automated generation tools highlight that attackers are no longer relying solely on manual exploit crafting; they are using AI to explore the vulnerability space at an unprecedented scale.

To survive this co-evolutionary arms race, defenders must adopt continuous, automated red-teaming and dynamic updating mechanisms. The macro-theme of this research intersection captures the adversarial arms race between sophisticated jailbreak attack frameworks and advanced alignment defenses, dictating that security cannot be a one-time post-training step. Instead, defenses must co-evolve. This involves using LLMs defensively to automatically generate novel attack vectors (internal red-teaming, as seen in PurpCode Liu et al. [2025a]) and using those vectors to continuously retrain and harden the model. By simulating the attacker's capabilities within the defense pipeline, security researchers can ensure that LLMs remain resilient not just against the threats of today, but against the automated, AI-generated exploits of tomorrow.

10 8. 7. Testing Methodologies and Evaluation

[Section not yet written — A mandatory, dedicated section synthesizing benchmarks, datasets, metrics, and evaluation frameworks]

11 9. 9. Discussion and Open Challenges

[Section not yet written — Synthesizes the gaps and limitations in the current LLM security landscape. It will discuss the fund]

12 10. 10. Conclusion

The rapid integration of Large Language Models (LLMs) into critical infrastructure has precipitated a paradigm shift in artificial intelligence, bringing unprecedented capabilities alongside profound security vulnerabilities. This survey has systematically mapped the expansive threat landscape of LLMs, categorizing vulnerabilities across the entire model lifecycle. During the training phase, the reliance on massive, uncurated datasets exposes models to sophisticated data and model poisoning attacks, compromising their foundational alignment and introducing dormant backdoors. At inference time, the attack surface widens considerably. We detailed how malicious actors exploit these systems through prompt injection, jailbreaking, and adversarial attacks, effectively bypassing safety guardrails to elicit harmful or unintended behaviors . Furthermore, we examined the intrinsic challenges of output integrity, particularly LLM hallucinations, which fundamentally undermine the reliability of these systems in high-stakes domains. In response to these threats, our review of cross-cutting defenses highlighted an ongoing, albeit challenging, transition from reactive patching to proactive, systemic mitigation strategies.

A critical synthesis of the literature reveals several overarching trends and patterns in LLM security. Most notably, there exists a persistent asymmetry between offensive and defensive capabilities. Attackers continuously innovate, leveraging automated and LLM-assisted techniques to generate sophisticated, adaptive exploits that consistently outpace static defensive measures . We also observe a distinct evolution in attack vectors, moving from simple, discrete textual manipulations to complex, continuous, and multimodal adversarial perturbations. As models evolve into Multimodal Large Language Models (MLLMs), the integration of visual and auditory inputs exponentially expands the attack surface, introducing cross-modal vulnerabilities that traditional text-based defenses fail to mitigate. Additionally, the "dual-use" nature of LLMs has become increasingly apparent; the very reasoning capabilities that make these models highly effective are routinely weaponized by adversaries to automate red-teaming and generate novel jailbreaks, creating a recursive and escalating security challenge.

The practical implications of these findings for LLM deployment are profound. For organizations integrating generative AI into commercial, healthcare, or critical

applications, security can no longer be treated as a secondary objective or a post-hoc patch. The pervasive nature of inference-time threats necessitates a defense-in-depth architecture. Relying solely on model alignment—such as Reinforcement Learning from Human Feedback (RLHF)—is demonstrably insufficient, as these safety mechanisms are frequently bypassed by sophisticated adversarial prompts. Consequently, deployments must incorporate robust input sanitization, dynamic output monitoring, and strict access controls. Furthermore, practitioners must navigate the inherent tension between model utility and security, often referred to as the "alignment tax." Overly aggressive defensive guardrails frequently degrade the model's performance on benign tasks, leading to high false-positive rates and a diminished user experience. Balancing these competing objectives remains a central, unresolved challenge in operationalizing LLMs.

Despite significant advancements in mitigation strategies, several critical open challenges demand immediate research attention. First, the theoretical limits of LLM alignment remain unresolved; it is currently unclear whether a mathematically provable defense against prompt injection and jailbreaking is achievable within the standard autoregressive transformer architecture. Second, the evaluation of LLM security is severely hindered by the lack of standardized, dynamic testing methodologies. Current static benchmarks are rapidly saturated or inadvertently included in training corpora, rendering them ineffective for evaluating true adversarial robustness. Future research must prioritize the development of dynamic, continuously updating evaluation frameworks that accurately reflect the evolving threat landscape.

Future work should also heavily invest in mechanistic interpretability. Understanding the internal representations and activation patterns that trigger malicious outputs is essential for developing targeted, intrinsic defenses, rather than relying on superficial input/output filters. The exploration of certified robustness techniques, adapted from traditional deep learning to the discrete and generative nature of LLMs, represents a highly promising, albeit mathematically challenging, frontier. Finally, as the field moves toward autonomous LLM agents capable of executing code and interacting with external APIs, securing the tool-use environments of these agents against indirect prompt injections will be paramount to preventing systemic compromises.

To navigate this complex security environment, we offer several actionable recommendations. For industry practitioners, we advocate for the implementation of continuous, automated red-teaming pipelines integrated directly into the MLOps lifecycle. Deployments should utilize multi-layered guardrails, isolating the LLM from direct execution privileges whenever possible, and employing secondary, specialized models to audit inputs and outputs for malicious intent. For the academic community, we recommend shifting focus from isolated, "whack-a-mole" vulnerability disclosures toward the development of systemic, scalable defense mechanisms and rigorous theoretical frameworks. Collaboration between open-source communities, industry leaders, and regulatory bodies is essential to establish standardized security protocols and vulnerability reporting mechanisms. Ultimately,

realizing the transformative potential of Large Language Models hinges not merely on scaling their parameters and capabilities, but on our collective ability to ensure their safety, integrity, and resilience against an increasingly sophisticated adversarial landscape.

---

13 References

See the accompanying `references.bib` *file for the complete bibliography.*

References

Ajwad Akil, Adrian Li, Imtiaz Karim, Arun Iyengar, Ashish Kundu, Vinny Parla, and Elisa Bertino. Llmalmorph: On the feasibility of generating variant malware using large-language-models. arXiv preprint, 2025.

Wes Bouaziz, Mathurin Videau, Nicolas Usunier, El Mahdi, and El Mhamdi. Winter soldier: Backdooring language models at pre-training with indirect data poisoning. arXiv preprint, 2025.

Yizhong Ding. A reward-driven automated webshell malicious-code generator for red-teaming. arXiv preprint, 2025.

Kaiwen Duan, Hongwei Yao, Yufei Chen, Ziyun Li, Tong Qiao, Zhan Qin, and Cong Wang. Badreward: Clean-label poisoning of reward models in text-to-image rlhf. *ArXiv*, abs/2506.03234, 2025.

Xiaojun Jia, Sensen Gao, Simeng Qin, Tianyu Pang, Chao Du, Yihao Huang, Xinfeng Li, Yiming Li, Bo Li, and Yang Liu. Adversarial attacks against closed-source mllms via feature optimal alignment. *ArXiv*, abs/2505.21494, 2025.

Jinxin Li, Shengyu Cheng, Junjie Hu, Jinting Wang, Rui Chen, Zhilong Zhou, and Dongbo Shan. Llm hallucination detection: A fast fourier transform method based on hidden layer temporal signals *. arXiv preprint, 2025.

Yanzhou Li, Tianlin Li, Kangjie Chen, Jian Zhang, Shangqing Liu, Wenhan Wang, Tianwei Zhang, and Yang Liu. Badedit: Backdooring large language models by model editing. *ArXiv*, abs/2403.13355, 2024a.

Yige Li, Hanxun Huang, Yunhan Zhao, Xingjun Ma, and Junfeng Sun. Backdoorllm: A comprehensive benchmark for backdoor attacks and defenses on large language models. 2024b.

Jiawei Liu, Nirav Diwan, Zhe Wang, Haoyu Zhai, Xiaona Zhou, Kiet Nguyen, Tianjiao Yu, Muntasir Wahed, Yinlin Deng, Hadjer Benkraouda, Yuxiang Wei,

Lingming Zhang, Ismini Lourentzou, and Gang Wang. Purpcode: Reasoning for safer code generation. arXiv preprint, 2025a.

Ting-Chun Liu, C. Hsu, Kuan-Yi Lee, C. Fu, and Hung yi Lee. Aegis : Automated co-evolutionary framework for guarding prompt injections schema. *ArXiv*, abs/2509.00088, 2025b.

Daniil Orel, Dilshod Azizov, Indraneil Paul, Yuxia Wang, Iryna Gurevych, and Preslav Nakov. Aicd bench: A challenging benchmark for ai-generated code detection. 2026.

Slms, Xiang Li, Chong Zhang, Jia Wang, Fangyu Wu, Yushi Li, and Xiaobo Jin. Efficient and stealthy jailbreak attacks via adversarial prompt distillation from llms to. arXiv preprint, 2025.

Maximilian Thang, Lichao Wu, Sasha Behrouzi, Mohamadreza Rostami, Jona Te Lintelo, Stjepan Picek, and Ahmad-Reza Sadeghi. Goodvibe: Security-by-vibe for llm-based code generation. arXiv preprint, 2026.

Xin Wang, Kai Chen, Xingjun Ma, Zhineng Chen, Jingjing Chen, and Yu-Gang Jiang. Advqdet: Detecting query-based adversarial attacks with adversarial contrastive prompt tuning. *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024.

Yanshu Wang, Shuaishuai Yang, Jingjing He, and Tong Yang. How few-shot demonstrations affect prompt-based defenses against llm jailbreak attacks. arXiv preprint, 2026.

Zhun Wang, Vincent Siu, Zhe Ye, Tianneng Shi, Yuzhou Nie, Xuandong Zhao, Chenguang Wang, Wenbo Guo, and D. Song. Agentvigil: Generic black-box red-teaming for indirect prompt injection against llm agents. 2025.

Yifan Yang, Qiao Jin, Furong Huang, and Zhiyong Lu. Adversarial attacks on large language models in medicine. *ArXiv*, 2024.

Xin Yin, Xinrui Li, Chao Ni, Xiaohu Yang, and Xiaodan Xu. Detecting llm-generated code with subtle modification by adversarial training. arXiv preprint, 2025.

Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. pages 7101–7117, 2025.

Zhifang Zhang, Qiqi Tao, Jiaqi Lv, Na Zhao, Lei Feng, and Joey Zhou. Tokenswap: Backdoor attack on the com-positional understanding of large vision-language models. arXiv preprint, 2025.